

Distributed Currency Exchange System

A Service-Oriented Architecture (SOA) Approach to Modern Forex Systems

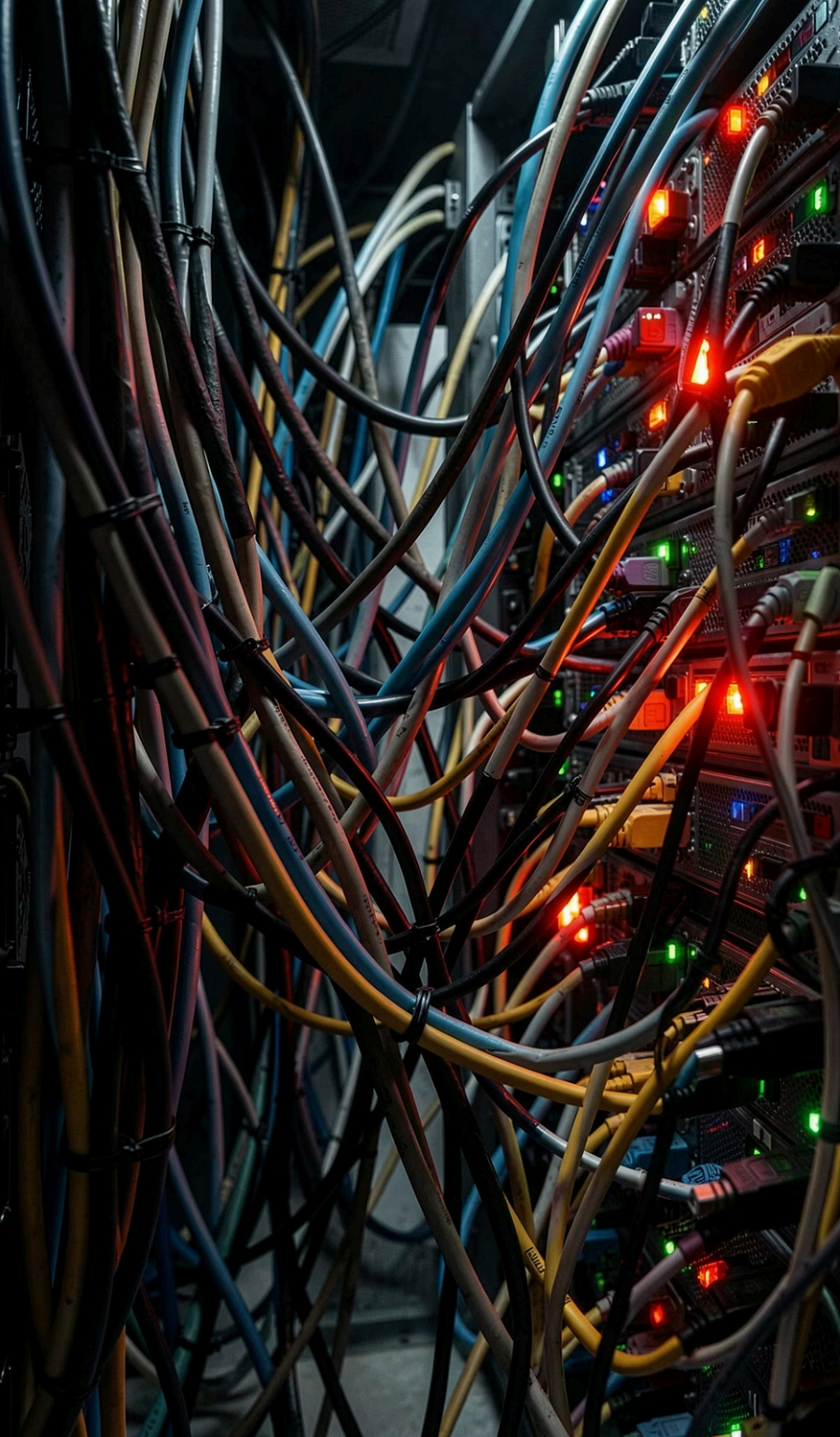
NETWORK APPLICATION DEVELOPMENT

SARKHAN RAHIMOV – ID: 63294

Building a secure, real-time, and scalable financial bridge between WCF services and WPF clients – a distributed system designed for the demands of modern currency exchange.



Made with GAMMA



The Challenge of Modern Exchange Systems

Traditional financial applications face critical architectural weaknesses when deployed across distributed networks. This project addresses three core problems that undermine reliability, security, and scalability.

Real-Time Data Accuracy

Financial decisions depend on live, precise exchange rates. Stale or manually entered data introduces costly errors and erodes user trust in the system.

Security Risks

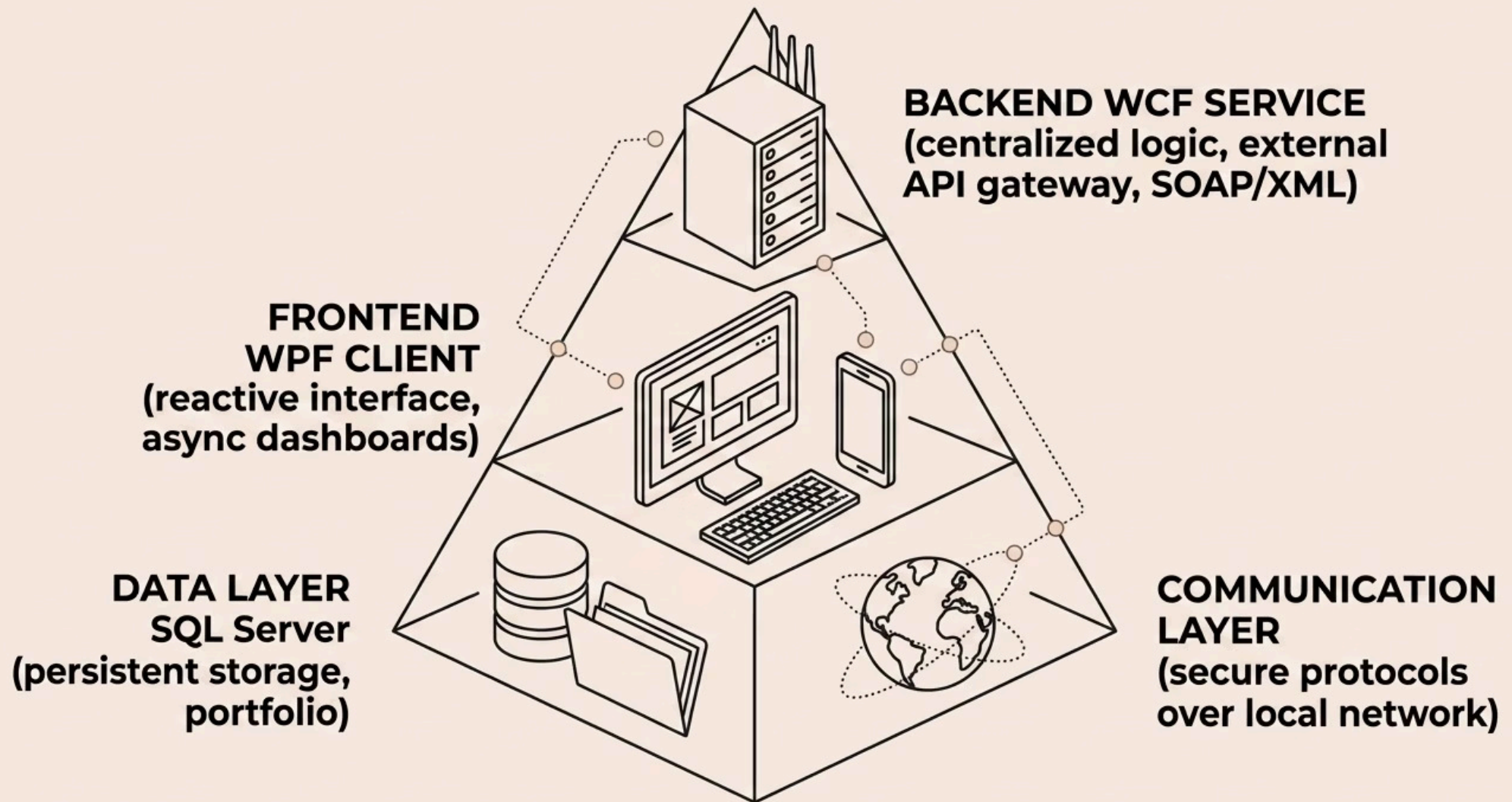
Direct database-to-client communication exposes sensitive financial data to interception and manipulation. A middle-tier service layer is essential to protect transactions.

The Vision: Separation of Concerns

By decoupling business logic, data access, and presentation layers within the .NET ecosystem, we achieve data integrity, high availability, and maintainable code.

Distributed Network Architecture

The system follows a clean multi-tier architecture designed for distributed environments. Each layer has a clearly defined responsibility, enabling independent scaling, testing, and maintenance.



Communication between layers is strictly governed: WCF services expose typed contracts over SOAP/XML, while external data from the National Bank of Poland is consumed via secure HTTPS/JSON – ensuring each channel is optimised for its purpose.

The Nervous System: WCF Service Layer

The Windows Communication Foundation service acts as the centralised backbone of the system – orchestrating all business logic, external API calls, and database interactions.

Service Contracts

Explicit `[ServiceContract]` and `[OperationContract]` interfaces define a predictable, versionable network API. Clients interact through strongly-typed proxies, eliminating ambiguity.

Server-Side Business Logic

Currency calculations, wallet top-ups, and transaction validation are all processed on the server. Clients never perform financial logic locally, preventing tampering.

Horizontal Scalability

A single WCF service host can serve multiple concurrent WPF clients. The stateless design allows the service to scale without modifying client-side code.

Client Interaction & UI Design

The WPF client delivers a polished, responsive financial dashboard built on modern UI/UX principles. Every interaction is designed to feel instantaneous, even when communicating over the network.

→ XAML-Powered Interface

Declarative XAML layouts enable rich data visualisation, custom controls, and consistent styling across the application.

→ Asynchronous Data Binding

All network calls execute on background threads via `async/await`, preventing UI freezing during service communication.

→ Proxy-Based Security

User credentials and transaction requests are validated exclusively through the WCF proxy – no sensitive logic resides on the client.



Real-Time Data Harvesting: NBP API

Live exchange rates are sourced directly from the **National Bank of Poland (NBP)**, eliminating manual input and ensuring every transaction reflects the official central bank rate at the moment of execution.

Integration Method

The WCF service establishes a secure **HTTPS/JSON** connection to the NBP public API. Rates are fetched on demand and cached briefly to balance freshness with performance.

Serialisation

Newtonsoft.Json handles high-performance deserialisation of API responses, mapping JSON payloads directly to strongly-typed C# data models.

Why Live Rates Matter

Fetching **Bid/Ask rates** directly from the central bank removes human error and ensures regulatory compliance. Users always trade at the official published rate – no spreads added by the application.

-  The NBP API is a publicly available, government-maintained endpoint – ideal for academic and production-grade financial applications requiring authoritative rate data.

Reliable Data Management: SQL Server

Financial operations demand absolute data integrity. Microsoft SQL Server provides the robust persistence layer required to track every user asset, transaction, and balance change with full auditability.

Transaction Integrity

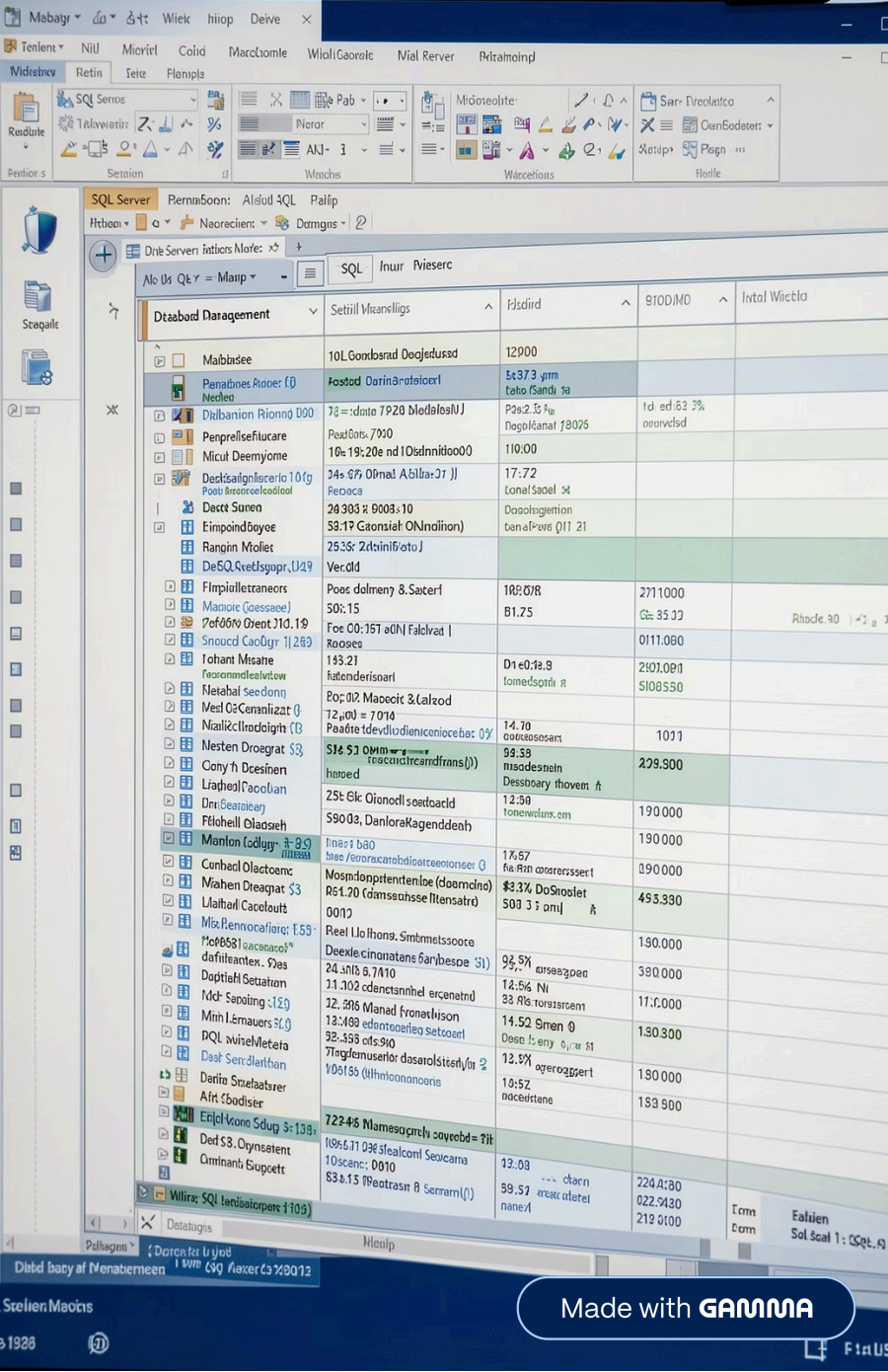
ADO.NET manages database connections with explicit transaction scopes. Every financial operation is **atomic** – either all changes commit successfully, or none do, preventing partial state corruption.

User Asset Tracking

Each user's **PLN balance** and currency portfolio are stored in normalised tables, updated in real-time as trades execute, and validated against business rules before persistence.

Audit History Ledger

Every operation – login, trade, top-up – is indexed with a timestamp and stored permanently. This ledger supports auditing, dispute resolution, and user-facing transaction history views.



Robustness & Error Handling

A distributed financial system must anticipate failure. The codebase enforces clean architecture patterns and defensive programming to ensure predictable behaviour under adverse network conditions.

1

Guard Clauses

A "flat" code structure eliminates deep conditional nesting. Invalid inputs are rejected at method entry, keeping core logic clean and testable.

2

Network Resilience

Common distributed errors – `TimeoutException`, `CommunicationException`, and `FaultException` – are caught and translated into user-friendly messages without crashing the client.

3

Input Sanitisation

Every user-supplied value is validated and sanitised before reaching the service layer. This prevents injection attacks and ensures only well-formed data enters the system.

4

Service Fault Contracts

WCF `FaultContract` attributes define structured error responses, allowing the client to handle service-side exceptions gracefully without exposing internal implementation details.

System Capabilities

The completed application delivers a comprehensive set of features that together form a fully functional distributed currency exchange platform – from onboarding to live trading and historical reporting.



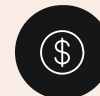
Instant Registration & Login

Secure account creation with credential validation via WCF. Sessions are managed server-side.



Real-Time PLN Balance

Live Polish Złoty balance management, updated instantly after every transaction or top-up operation.



Live Currency Trading

Buy and sell foreign currencies at official NBP rates. Trades execute atomically with immediate balance updates.



Synchronised Transaction History

A complete, indexed ledger of all user operations – accessible in real-time and consistent across sessions.



Auto Data Synchronisation

The WPF client automatically refreshes rates and balances from the WCF service, ensuring the UI always reflects the current system state.

Results & Future Enhancements

This project successfully demonstrates a fully functional, distributed network application that adheres to **OOP principles** and **SOA standards** – delivering a secure, real-time currency exchange experience across the .NET ecosystem.

What Was Achieved

- Multi-tier SOA architecture with clear separation of concerns
- Live NBP API integration with JSON serialisation
- Atomic SQL Server transactions with full audit logging
- Responsive WPF client with async data binding
- Robust error handling across the distributed stack

Future Scope

- **Mobile Client Support** – Extending the WCF service to Xamarin-based iOS and Android applications
- **Cryptocurrency Integration** – Adding live crypto API feeds alongside fiat currency rates
- **RESTful Migration** – Exploring a transition from SOAP to REST/JSON for broader interoperability

"Reliability in Every Transaction, Precision in Every Rate."